Python Basic Syntax

What is Syntax?

The set of rules that define how Python code should be written. Think of it like grammar in a language—if you don't follow it, Python won't understand you!

Key Rules to Know:

Comments (#) – Comments are like sticky notes in your code. They start with # and are ignored by Python, so they're just for you or others to understand what the code does.
Example: # This calculates your age
Indentation (Spaces/Tabs) – Python uses them to group code (like paragraphs in a book).
Code Blocks – Sections of code that belong together (e.g., inside loops or functions).
Keywords – Special words Python reserves (e.g., if, for, while). Don't use these as variable names!

Input/Output

print("Hello") → Displays text on screen.
input("What's your name? ") → Asks the user to type something.

Comment

[ ]:
```
# This is a comment it does not execute
## Thhis is also  comment it does not execute
```

Indentation

[ ]:
```
# Indentation (spaces/Tabs)
number = 5
if number > 0:
    print("The number is positive!")  # Indented with 4 spaces
```

Code Block

[ ]:
```
# Code blocks
# A code block is a group of lines that Python runs together
for i in range(1, 4):
    print(i)
```

[ ]:
```
# Code Blocks
# A code block is a group of lines that Python runs together
number = 5
while number > 0:
    print(number)
    number -= 1  # Decrease number by 1
```

Python Reserve Word

[ ]:
```
# keywords
# Special words Python reserves

if
else
elif
for
in  # etc   Don't use these as variable names
```

Output

[ ]:
```
# Output
print('Hello, World!')  # This prints a string to  the console
```

f-string

An f-string (short for formatted string literal) is a way to embed expressions inside string literals in Python. It was introduced in Python 3.6 and makes string formatting more readable and concise.

Syntax: You write an f-string by prefixing the string with the letter f or F and then including variables or expressions inside curly braces {}.

[ ]:
```
# OutPut

name = "Alice"
print(f'hello {name}!')
```

Input

[ ]:
```
# Input
name = input("Enter Your Name:") # This get input from the user
print("Hello, " + name) # This prints a greeeting with your name entered
```

[ ]:
```
# Input
age = int(input("Enter your age: "))  # Get age as an integer
print(f"You are {age} years old!")  # Print the age
```

Variables

What's a Variable?

A named container that stores data (like a labeled box).
Example: score = 100 → Stores the number 100 in a variable named score.

Naming Rules:

Start with a letter or _ (not a number).
Can contain letters, numbers, or _ (no spaces or symbols).
Case-sensitive (Name ≠ name).
Avoid Python keywords (e.g., don't name a variable print).

Assignment (=):

fruit = "apple" → Puts "apple" into the variable fruit.
Can update later: fruit = "banana".

Variable

[ ]:
```
# Variables
name = "John" # This is s string variable
age = 25 # This is an integer variable
height = 5.7 # This id a float Variable
_is_valid = True  # This is also a valid variable name (starts with an underscore)
```

```
True
```

Variable Case Sensivity

[18]:
```
# Case Sensitivity
Name = "Alice"
name = 'alice'  # Name and name are different variables
```

Variable Assignment

[ ]:
```
# Assignment (=)
x = 10  # Assigns the value 10 to the variable x
y = 5   # Assigns the value 5 to the variable y

# Multiple Assignments
x, y = 10, 5  # Assigns 10 to x and 5 to y in one line
```

Updating Variable

[17]:
```
#update variable
name = "Alice"
name = "Bob"  # Updates the variable name to "Bob"

age = 30
age = 25  # Updates the variable age to 25
```

Data Types

What's a Data Type?

A Python data type specifies the category of data a variable can store, determining its behavior and allowed operations.

Common Types:

Text Data Type

[ ]:
```
# Text Data Types
# Also known as strings, are used to represent text

place = "Lagos"  # A string variable
country = "Nigeris"  # Another string variable
```

Numeric Data Type

[ ]:
```
# Numberoc Data Types
number = 10  # An integer variable
hieght = 4.7 # A float variable
```

Boolen Data Type

[ ]:
```
# Boolean Data Types

is_valid = True  # A boolean variable (True )
invalid = False  # A boolean variable (False)
```

Sequence Data Type

[ ]:
```
# Sequence Types

list_example = [1, 2, 3, 4, 5]  # A list is an ordered collection of items

tuple_example = (1, 2, 3)  # A tuple is an ordered collection of items, but immutable


dictionary_example = {"key1": "value1", "key2": "value2"}  # A dictionary is a collection of key-value pairs

set_example = {1, 2, 3}  # A set is an unordered collection of unique items
```

None Data Type

[31]:
```
# None Data Type

none_value = None  # Represents the absence of a value or a null value
```

Checking Data Type

[ ]:
```
# Checking Data Type

name = "Alice"
print(type(name))  # <class 'str'>, this is a string


age = 30
print(type(age))  # <class 'int'>, this is an integer


names = ["Alice", "Bob", "Charlie"]
print(type(names))  # <class 'list'>, this is a list


data = {'name': 'Alice', 'age': 30}
print(type(data))  # <class 'dict'>, this is a dictionary
```

Converting Data Type

[ ]:
```
# Converting Data Types

age_str = '30'
age = int(age_str)  # Converts string to integer
print(age)  # Output: 30


number = 100
number_str = str(number)  # Converts integer to string
print(number_str)  # Output: '100'
```

```
30
```